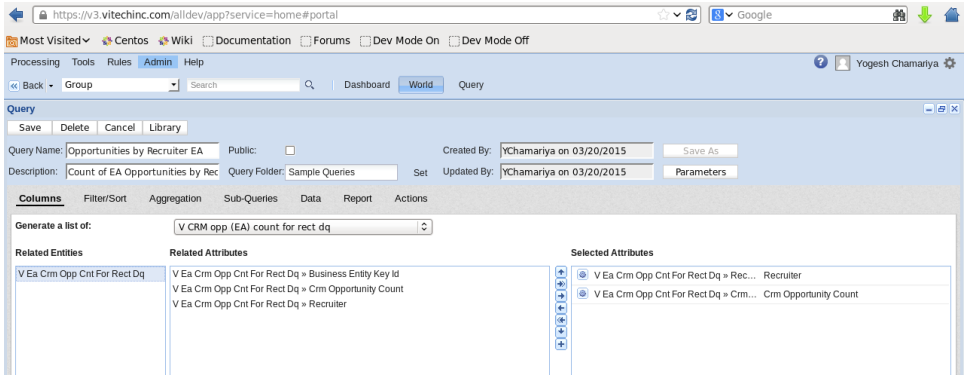
Query Module

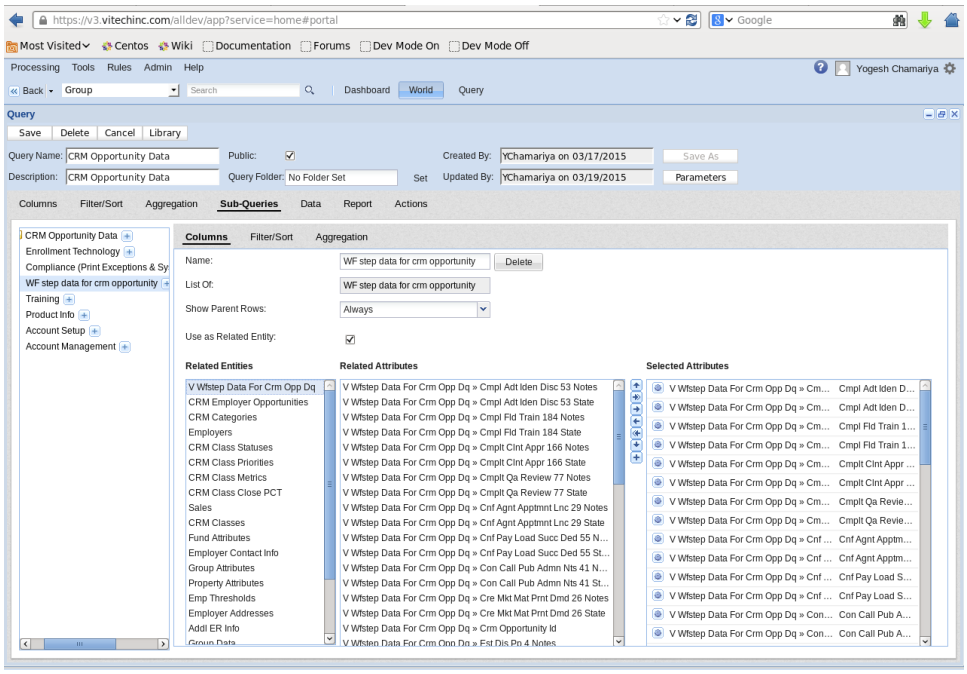
Adding client views in Query Module

- Create views in **client** schema
- View Name format : **V_<<VIEW_NAME>>_DQ** (DQ stands for Data Query)
- Column names in View should be **readable** as they will be converted into column names in query module.
    - Example : CRM_OPPORTUNITY_ID will be converted to Crm Opportunity Id
- View should have **one column** which can be joined to an entity for which you are writing the view
- Views can be added as **Related entities** or **Sub-queries**

    - **Related entities**



    - **Subqueries** (Mostly used in documents to display grid)



- **View should be added as related entity** if we have one-to-one mapping between View and existing Related entity
- **View should be added as sub-queries** if we have one to many mapping between existing Related entity and View

Scripts used to add a view:

- Create the view and for example lets say the view name is V_EA_CRM_OPP_CNT_FOR_RECT_DQ which was created in client schema

- Load catalog tables

```
--load sys tables

BEGIN

   dbo.pkg_benefit_ddl.p_load_sys_tables('N','V_EA_CRM_OPP_CNT_FOR_RECT_DQ');

  COMMIT;

END;
/
```

- Update entity query visible to Y

```
--update entity query visble to Y

begin

  update dbo.entity e set e.QUERY_VISIBLE='Y' where e.ENTITY_NAME='V_EA_CRM_OPP_CNT_FOR_RECT_DQ';

  commit;

end;

/
```

- set entity attributes visible_flag and query_visible to Y

```
BEGIN

update dbo.entity_attribute ea

  set ea.QUERY_VISIBLE = 'Y',

      ea.VISIBLE_FLAG = 'Y'

where ea.ENTITY_ID = dbo.pkg_benefit_s.f_get_entity_id('V_EA_CRM_OPP_CNT_FOR_RECT_DQ');

COMMIT;

END;

/
```

- Create a query record for the view (Make sure this one is re-runnable)

```
--add query
begin
insert into dbo.query
  (query_id,
   entity_id,
   attribute_id,
   display_name,
   internal_name,
   product_flag,
   browse_default)
values
  (dbo.s_query_pk.nextval,
   dbo.pkg_benefit_s.f_get_entity_id('V_EA_CRM_OPP_CNT_FOR_RECT_DQ'),
   dbo.pkg_benefit_s.f_get_attribute_id('BUSINESS_ENTITY_KEY_ID','ID'),
   'V CRM opp (EA) count for rect dq',
   'V_EA_CRM_OPP_CNT_FOR_RECT_DQ',
   'N',
   'Y');
  commit;
end;
/
```

- Add joins between related entities and the view you created (view may already exist in the system so you do not need to create it from scratch).
    - If the data in client view is optional in regards to the "main" entity, the join operator should be '(+)=' instead of '='

```
--add joins for related entity
--prime_entity_name,left_attribute_name and left_entity_name will be related entity
begin
web.pkg_util_query.p_update_joins
  (piv_prime_entity_name    => 'V310_ENTITY'
  ,piv_second_entity_name   => 'V_EA_CRM_OPP_CNT_FOR_RECT_DQ'
  ,pin_seq_no               => 1
  ,piv_join_expression      => null
  ,piv_operator             => '='
  ,piv_left_attribute_name  => 'BUSINESS_ENTITY_KEY_ID'
  ,piv_left_entity_name     => 'V_EA_CRM_OPP_CNT_FOR_RECT_DQ'
  ,piv_right_attribute_name => 'BUSINESS_ENTITY_KEY_ID'
  ,piv_right_entity_name    => 'V310_ENTITY'
  ,piv_join_name            => null
);
  commit;
end;
/
```

```
--add joins as sub-queries
--prime_entity_name,left_attribute_name and left_entity_name will be view
begin
web.pkg_util_query.p_update_joins
  (piv_prime_entity_name    => 'V_EA_CRM_OPP_CNT_FOR_RECT_DQ'
  ,piv_second_entity_name   => 'V310_ENTITY'
  ,pin_seq_no               => 1
  ,piv_join_expression      => null
  ,piv_operator             => '='
  ,piv_left_attribute_name  => 'BUSINESS_ENTITY_KEY_ID'
  ,piv_left_entity_name     => 'V310_ENTITY'
  ,piv_right_attribute_name => 'BUSINESS_ENTITY_KEY_ID'
  ,piv_right_entity_name    => 'V_EA_CRM_OPP_CNT_FOR_RECT_DQ'
  ,piv_join_name            => null
);
  commit;
end;
/
```

- **Examples on allstate instance**
  - Related entity example : Opportunities by Recruiter NAT
  - Sub-queries example : CRM Opportunity Data

Extracting a query module (This steps need to performed in order):

- Need to place scripts created for views in queue (Create public queries by checking public flag)
- Need all the sql scripts which are used for adding views to related entities or sub-queries in the queue
- Extract query into ASW and place it queue. (Query needs to be public)

NOTE:

1. Sub-queries can be converted into related entities by checking '**Use a related entity**' in sub-Queries tab but this should be done only when your sub-query returns single row.
2. If view has a column wrapped in a function (such as contact) then make sure to cast that column to any of the existing attribute types in v3 or else that column will not be visible in query module.
3. At this point Joins should be added between views. Example: V310_ENTITY and V_EA_CRM_OPP_CNT_FOR_RECT_DQ
4. **Every time after creating/modifying a view for the query module**, the first steps you should take are:

    a. Refresh catalog from UI (Admin -> Utilities -> Refresh Catalog (button))
    b. Refresh browser page (Ctrl + R) or log out and log back into the app

Sample Code:

https://v3.vitechinc.com/crucible/cru/ALLSTATE-271

Please feel free to add or remove content.

Auto Generating Views for Entities (BUSINESS_ENTITY_SUBTYPE)

Use dbo.pkg_ben_view_gen.p_build_bus_ent_subtype_view to create views.
UI does call the proc when the flag changes . So it can be done through UI also , but involves flipping twice , if query visible is Y already.

Refer VTHREE-20618 for more details.

Example:

```
begin
dbo.pkg_ben_view_gen.p_build_bus_ent_subtype_view(pin_bus_ent_subtype_id => 294);
end;
```

Query to find joins:

```
select j.joins_id
    , ep.entity_name                        as prim
    , es.entity_name                        as sec
    , j.seq_no                              as seq
    , el.entity_name ||
      case
        when el.entity_name is not null then '.'
        else to_char(null)
      end || al.attribute_name              as left
    , er.entity_name ||
      case
        when er.entity_name is not null then '.'
        else to_char(null)
      end || ar.attribute_name              as right
    , j.operator
    , j.join_expression                     as express
    , j.joins_id
    , j.join_name
    , j.show_as_related
    , j.show_as_subquery
    , j.inserted_date
    , j.updated_date
 from dbo.joins      j
    , dbo.entity     ep
    , dbo.entity     es
    , dbo.entity     el
    , dbo.attribute  al
    , dbo.entity     er
    , dbo.attribute  ar
where dbo.pkg_benefit_s.f_get_entity_id(upper(trim('V310_BUSINESS_ENTITY_RELATION')))
      in (j.primary_entity_id,j.secondary_entity_id)
  and ep.entity_id       = j.primary_entity_id
  and es.entity_id       = j.secondary_entity_id
  and el.entity_id       = j.left_entity_id
  and al.attribute_id    = j.left_attribute_id
  and er.entity_id(+)    = j.right_entity_id
  and ar.attribute_id(+) = j.right_attribute_id
 order by 2,3,4
```